in parallel to produce the output representations which are concatenated and once again projected:

$$\begin{aligned} \texttt{MH}(\boldsymbol{H}^l) &= [\text{head}_1; \text{head}_2; \ldots; \text{head}_h]\boldsymbol{W}^O \\ \text{head}_i &= \texttt{Attention}\big(\boldsymbol{H}^l\boldsymbol{W}_i^Q, \boldsymbol{H}^l\boldsymbol{W}_i^K, \boldsymbol{H}^l\boldsymbol{W}_i^V\big) \end{aligned} \tag{1}$$

where the projections matrices for each head $\boldsymbol{W}_i^Q \in \mathbb{R}^{d\times d/h}$, $\boldsymbol{W}_i^K \in \mathbb{R}^{d\times d/h}$, $\boldsymbol{W}_i^V \in \mathbb{R}^{d\times d/h}$, and $\boldsymbol{W}_i^O \in \mathbb{R}^{d\times d}$ are learnable parameters. Here, we omit the layer subscript l for the sake of simplicity. In fact, these projection parameters are not shared across the layers. Here, the `Attention` function is *Scaled Dot-Product Attention*:

$$\texttt{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d/h}}\right)\boldsymbol{V} \tag{2}$$

where *query* $\boldsymbol{Q}$, *key* $\boldsymbol{K}$, and *value* $\boldsymbol{V}$ are projected from the same matrix $\boldsymbol{H}^l$ with different learned projection matrices as in Equation 1. The *temperature* $\sqrt{d/h}$ is introduced to produce a softer attention distribution for avoiding extremely small gradients [16, 52].

Position-wise Feed-Forward Network. As described above, the self-attention sub-layer is mainly based on linear projections. To endow the model with nonlinearity and interactions between different dimensions, we apply a *Position-wise Feed-Forward Network* to the outputs of the self-attention sub-layer, separately and identically at each position. It consists of two affine transformations with a Gaussian Error Linear Unit (`GELU`) activation in between:

$$\begin{aligned} \texttt{PFFN}(\boldsymbol{H}^l) &= \left[\texttt{FFN}(\boldsymbol{h}_1^l)^\top; \ldots; \texttt{FFN}(\boldsymbol{h}_t^l)^\top\right]^\top \\ \texttt{FFN}(\boldsymbol{x}) &= \texttt{GELU}\big(\boldsymbol{x}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)}\big)\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)} \\ \texttt{GELU}(x) &= x\Phi(x) \end{aligned} \tag{3}$$

where $\Phi(x)$ is the cumulative distribution function of the standard gaussian distribution, $\boldsymbol{W}^{(1)} \in \mathbb{R}^{d\times 4d}$, $\boldsymbol{W}^{(2)} \in \mathbb{R}^{4d\times d}$, $\boldsymbol{b}^{(1)} \in \mathbb{R}^{4d}$ and $\boldsymbol{b}^{(2)} \in \mathbb{R}^d$ are learnable parameters and shared across all positions. We omit the layer subscript l for convenience. In fact, these parameters are different from layer to layer. In this work, following OpenAI GPT [38] and BERT [6], we use a smoother `GELU` [13] activation rather than the standard `ReLu` activation.

Stacking Transformer Layer. As elaborated above, we can easily capture item-item interactions across the entire user behavior sequence using self-attention mechanism. Nevertheless, it is usually beneficial to learn more complex item transition patterns by stacking the self-attention layers. However, the network becomes more difficult to train as it goes deeper. Therefore, we employ a residual connection [9] around each of the two sub-layers as in Figure 1a, followed by layer normalization [1]. Moreover, we also apply dropout [47] to the output of each sub-layer, before it is normalized. That is, the output of each sub-layer is $\texttt{LN}(\boldsymbol{x} + \texttt{Dropout}(\text{sublayer}(\boldsymbol{x})))$, where $\text{sublayer}(\cdot)$ is the function implemented by the sub-layer itself, `LN` is the layer normalization function defined in [1]. We use `LN` to normalize the inputs over all the hidden units in the same layer for stabilizing and accelerating the network training.

In summary, BERT4Rec refines the hidden representations of each layer as follows:

$$\boldsymbol{H}^l = \texttt{Trm}\big(\boldsymbol{H}^{l-1}\big), \quad \forall i \in [1, \ldots, L] \tag{4}$$

$$\texttt{Trm}(\boldsymbol{H}^{l-1}) = \texttt{LN}\Big(\boldsymbol{A}^{l-1} + \texttt{Dropout}\big(\texttt{PFFN}(\boldsymbol{A}^{l-1})\big)\Big) \tag{5}$$

$$\boldsymbol{A}^{l-1} = \texttt{LN}\Big(\boldsymbol{H}^{l-1} + \texttt{Dropout}\big(\texttt{MH}(\boldsymbol{H}^{l-1})\big)\Big) \tag{6}$$

3.4 Embedding Layer

As elaborated above, without any recurrence or convolution module, the Transformer layer `Trm` is not aware of the order of the input sequence. In order to make use of the sequential information of the input, we inject *Positional Embeddings* into the input item embeddings at the bottoms of the Transformer layer stacks. For a given item v_i, its input representation $\boldsymbol{h}_i^0$ is constructed by summing the corresponding item and positional embedding:

$$\boldsymbol{h}_i^0 = \boldsymbol{v}_i + \boldsymbol{p}_i$$

where $\boldsymbol{v}_i \in \boldsymbol{E}$ is the $d-$dimensional embedding for item v_i, $\boldsymbol{p}_i \in \boldsymbol{P}$ is the $d-$dimensional positional embedding for position index i. In this work, we use the learnable positional embeddings instead of the fixed sinusoid embeddings in [52] for better performances. The positional embedding matrix $\boldsymbol{P} \in \mathbb{R}^{N\times d}$ allows our model to identify which portion of the input it is dealing with. However, it also imposes a restriction on the maximum sentence length N that our model can handle. Thus, we need to truncate the the input sequence $[v_1, \ldots, v_t]$ to the last N items $[v_{t-N+1}^u, \ldots, v_t]$ if $t > N$.

3.5 Output Layer

After L layers that hierarchically exchange information across all positions in the previous layer, we get the final output $\boldsymbol{H}^L$ for all items of the input sequence. Assuming that we mask the item v_t at time step t, we then predict the masked items v_t base on $\boldsymbol{h}_t^L$ as shown in Figure 1b. Specifically, we apply a two-layer feed-forward network with `GELU` activation in between to produce an output distribution over target items:

$$P(v) = \text{softmax}\big(\texttt{GELU}(\boldsymbol{h}_t^L\boldsymbol{W}^P + \boldsymbol{b}^P)\boldsymbol{E}^\top + \boldsymbol{b}^O\big) \tag{7}$$

where $\boldsymbol{W}^P$ is the learnable projection matrix, $\boldsymbol{b}^P$, and $\boldsymbol{b}^O$ are bias terms, $\boldsymbol{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is the embedding matrix for the item set $\mathcal{V}$. We use the shared item embedding matrix in the input and output layer for alleviating overfitting and reducing model size.

3.6 Model Learning

Training. Conventional unidirectional sequential recommendation models usually train the model by predicting the next item for each position in the input sequence as illustrated in Figure 1c and 1d. Specifically, the target of the input sequence $[v_1, \ldots, v_t]$ is a shifted version $[v_2, \ldots, v_{t+1}]$. However, as shown in Figure 1b, jointly conditioning on both left and right context in a bidirectional model would cause the final output representation of each item to contain the information of the target item. This makes predicting the future become trivial and the network would not learn anything useful. A simple solution for this issue is to create $t-1$ samples (subsequences with next items like $([v_1], v_2)$ and $([v_1, v_2], v_3)$) from the original length t behavior sequence and then encode each historical

subsequence with the bidirectional model to predict the target item. However, this approach is very time and resources consuming since we need to create a new sample for each position in the sequence and predict them separately.

In order to efficiently train our proposed model, we apply a new objective: *Cloze* task [50] (also known as "Masked Language Model" in [6]) to sequential recommendation. It is a test consisting of a portion of language with some words removed, where the participant is asked to fill the missing words. In our case, for each training step, we randomly mask ρ proportion of all items in the input sequence (*i.e.*, replace with special token "[mask]"), and then predict the original ids of the masked items based solely on its left and right context. For example:

$$\textbf{Input}: [v_1, v_2, v_3, v_4, v_5] \xrightarrow{\text{randomly mask}} [v_1, \texttt{[mask]}_1, v_3, \texttt{[mask]}_2, v_5]$$
$$\textbf{Labels}: \texttt{[mask]}_1 = v_2, \quad \texttt{[mask]}_2 = v_4$$

The final hidden vectors corresponding to "[mask]" are fed into an output softmax over the item set, as in conventional sequential recommendation. Eventually, we define the loss for each masked input $\mathcal{S}'_u$ as the negative log-likelihood of the masked targets:

$$\mathcal{L} = \frac{1}{|\mathcal{S}_u^m|} \sum_{v_m \in \mathcal{S}_u^m} -\log P(v_m = v_m^* | \mathcal{S}'_u) \tag{8}$$

where $\mathcal{S}'_u$ is the masked version for user behavior history $\mathcal{S}_u$, $\mathcal{S}_u^m$ is the random masked items in it, v_m^* is the true item for the masked item v_m, and the probability $P(\cdot)$ is defined in Equation 7.

An additional advantage for Cloze task is that it can generate more samples to train the model. Assuming a sequence of length n, conventional sequential predictions in Figure 1c and 1d produce n unique samples for training, while BERT4Rec can obtain $\binom{n}{k}$ samples (if we randomly mask k items) in multiple epochs. It allows us to train a more powerful bidirectional representation model.

Test. As described above, we create a mismatch between the training and the final sequential recommendation task since the Cloze objective is to predict the current masked items while sequential recommendation aims to predict the future. To address this, we append the special token "[mask]" to the end of user's behavior sequence, and then predict the next item based on the final hidden representation of this token. To better match the sequential recommendation task (*i.e.*, predict the last item), we also produce samples that only mask the last item in the input sequences during training. It works like fine-tuning for sequential recommendation and can further improve the recommendation performances.

3.7 Discussion

Here, we discuss the relation of our model with previous related work.

SASRec. Obviously, SASRec is a left-to-right unidirectional version of our BERT4Rec with single head attention and causal attention mask. Different architectures lead to different training methods. SASRec predicts the next item for each position in a sequence, while BERT4Rec predicts the masked items in the sequence using Cloze objective.

CBOW & SG. Another very similar work is Continuous Bag-of-Words (CBOW) and Skip-Gram (SG) [35]. CBOW predicts a target word using the average of all the word vectors in its context (both left and right). It can be seen as a simplified case of BERT4Rec, if we use one self-attention layer in BERT4Rec with uniform attention weights on items, unshare item embeddings, remove the positional embedding, and only mask the central item. Similar to CBOW, SG can also be seen as a simplified case of BERT4Rec following similar reduction operations (mask all items except only one). From this point of view, Cloze can be seen as a general form for the objective of CBOW and SG. Besides, CBOW uses a simple aggregator to model word sequences since its goal is to learn good word representations, not sentence representations. On the contrary, we seek to learn a powerful behavior sequence representation model (deep self-attention network in this work) for making recommendations.

BERT. Although our BERT4Rec is inspired by the BERT in NLP, it still has several differences from BERT: ***a)*** The most critical difference is that BERT4Rec is an end-to-end model for sequential recommendation, while BERT is a pre-training model for sentence representation. BERT leverages large-scale task-independent corpora to pre-train the sentence representation model for various text sequence tasks since these tasks share the same background knowledge about the language. However, this assumption does not hold in the recommendation tasks. Thus we train BERT4Rec end-to-end for different sequential recommendation datasets. ***b)*** Different from BERT, we remove the next sentence loss and segment embeddings since BERT4Rec models a user's historical behaviors as only one sequence in sequential recommendation task.

Table 1: Statistics of datasets.

Datasets	#users	#items	#actions	Avg. length	Density
Beauty	40,226	54,542	0.35m	8.8	0.02%
Steam	281,428	13,044	3.5m	12.4	0.10%
ML-1m	6040	3416	1.0m	163.5	4.79%
ML-20m	138,493	26,744	20m	144.4	0.54%

4 EXPERIMENTS

4.1 Datasets

We evaluate the proposed model on four real-world representative datasets which vary significantly in domains and sparsity.

- Amazon **Beauty**[3]: This is a series of product review datasets crawled from Amazon.com by McAuley et al. [34]. They split the data into separate datasets according to the top-level product categories on Amazon. In this work, we adopt the "Beauty" category.
- **Steam**[4]: This is a dataset collected from Steam, a large online video game distribution platform, by Kang and McAuley [22].
- **MovieLens** [8]: This is a popular benchmark dataset for evaluating recommendation algorithms. In this work, we adopt two well-established versions, MovieLens 1m (**ML-1m**)[5] and MovieLens 20m (**ML-20m**)[6].

For dataset preprocessing, we follow the common practice in [22, 40, 49]. For all datasets, we convert all numeric ratings or the presence of a review to implicit feedback of 1 (*i.e.*, the user interacted

[3] http://jmcauley.ucsd.edu/data/amazon/
[4] https://cseweb.ucsd.edu/~jmcauley/datasets.html#steam_data
[5] https://grouplens.org/datasets/movielens/1m/
[6] https://grouplens.org/datasets/movielens/20m/

with the item). After that, we group the interaction records by users and build the interaction sequence for each user by sorting these interaction records according to the timestamps. To ensure the quality of the dataset, following the common practice [12, 22, 40, 49], we keep users with at least five feedbacks. The statistics of the processed datasets are summarized in Table 1.

4.2 Task Settings & Evaluation Metrics

To evaluate the sequential recommendation models, we adopted the *leave-one-out* evaluation (*i.e.*, next item recommendation) task, which has been widely used in [12, 22, 49]. For each user, we hold out the last item of the behavior sequence as the test data, treat the item just before the last as the validation set, and utilize the remaining items for training. For easy and fair evaluation, we follow the common strategy in [12, 22, 49], pairing each ground truth item in the test set with 100 randomly sampled *negative* items that the user has not interacted with. To make the sampling reliable and representative [19], these 100 negative items are sampled according to their popularity. Hence, the task becomes to rank these negative items with the ground truth item for each user.

Evaluation Metrics. To evaluate the ranking list of all the models, we employ a variety of evaluation metrics, including *Hit Ratio* (HR), *Normalized Discounted Cumulative Gain* (NDCG), and *Mean Reciprocal Rank* (MRR). Considering we only have one ground truth item for each user, HR@k is equivalent to Recall@k and proportional to Precision@k; MRR is equivalent to Mean Average Precision (MAP). In this work, we report HR and NDCG with $k = 1, 5, 10$. For all these metrics, the higher the value, the better the performance.

4.3 Baselines & Implementation Details

To verify the effectiveness of our method, we compare it with the following representative baselines:

- **POP**: It is the simplest baseline that ranks items according to their popularity judged by the number of interactions.
- **BPR-MF** [39]: It optimizes the matrix factorization with implicit feedback using a pairwise ranking loss.
- **NCF** [12]: It models userâĂŞitem interactions with a MLP instead of the inner product in matrix factorization.
- **FPMC** [40]: It captures users' general taste as well as their sequential behaviors by combing MF with first-order MCs.
- **GRU4Rec** [15]: It uses GRU with ranking based loss to model user sequences for session based recommendation.
- **GRU4Rec$^+$** [14]: It is an improved version of GRU4Rec with a new class of loss functions and sampling strategy.
- **Caser** [49]: It employs CNN in both horizontal and vertical way to model high-order MCs for sequential recommendation.
- **SASRec** [22]: It uses a left-to-right Transformer language model to capture users' sequential behaviors, and achieves state-of-the-art performance on sequential recommendation.

For NCF[7], GRU4Rec[8], GRU4Rec$^+$[8], Caser[9], and SASRec[10], we use code provided by the corresponding authors. For BPR-MF and FPMC, we implement them using `TensorFlow`. For common hyper-parameters in all models, we consider the hidden dimension size d from $\{16, 32, 64, 128, 256\}$, the ℓ_2 regularizer from {1, 0.1, 0.01, 0.001, 0.0001}, and dropout rate from $\{0, 0.1, 0.2, \cdots, 0.9\}$. All other hyper-parameters (*e.g.*, Markov order in Caser) and initialization strategies are either followed the suggestion from the methods' authors or tuned on the validation sets. We report the results of each baseline under its optimal hyper-parameter settings.

We implement BERT4Rec[11] with `TensorFlow`. All parameters are initialized using truncated normal distribution in the range $[-0.02, 0.02]$. We train the model using Adam [24] with learning rate of 1e-4, $\beta_1 = 0.9, \beta_2 = 0.999$, ℓ_2 weight decay of 0.01, and linear decay of the learning rate. The gradient is clipped when its ℓ_2 norm exceeds a threshold of 5. For fair comparison, we set the layer number $L = 2$ and head number $h = 2$ and use the same maximum sequence length as in [22], $N = 200$ for ML-1m and ML-20m, $N = 50$ for Beauty and Steam datasets. For head setting, we empirically set the dimensionality of each head as 32 (single head if $d < 32$). We tune the mask proportion ρ using the validation set, resulting in $\rho = 0.6$ for Beauty, $\rho = 0.4$ for Steam, and $\rho = 0.2$ for ML-1m and ML-20m. All the models are trained from scratch without any pre-training on a single NVIDIA GeForce GTX 1080 Ti GPU with a batch size of 256.

4.4 Overall Performance Comparison

Table 2 summarized the best results of all models on four benchmark datasets. The last column is the improvements of BERT4Rec relative to the best baseline. We omit the NDCG@1 results since it is equal to HR@1 in our experiments. It can be observed that:

The non-personalized POP method gives the worst performance[12] on all datasets since it does not model user's personalized preference using the historical records. Among all the baseline methods, sequential methods (*e.g.*, FPMC and GRU4Rec$^+$) outperforms non-sequential methods (*e.g.*, BPR-MF and NCF) on all datasets consistently. Compared with BPR-MF, the main improvement of FPMC is that it models users' historical records in a sequential way. This observation verifies that considering sequential information is beneficial for improving performances in recommendation systems.

Among sequential recommendation baselines, Caser outperforms FPMC on all datasets especially for the dense dataset ML-1m, suggesting that high-order MCs is beneficial for sequential recommendation. However, high-order MCs usually use very small order L since they do not scale well with the order L. This causes Caser to perform worse than GRU4Rec$^+$ and SASRec, especially on sparse datasets. Furthermore, SASRec performs distinctly better than GRU4Rec and GRU4Rec$^+$, suggesting that self-attention mechanism is a more powerful tool for sequential recommendation.

According to the results, it is obvious that BERT4Rec performs best among all methods on four datasets in terms of all evaluation metrics. It gains **7.24%** HR@10, **11.03%** NDCG@10, and **11.46%** MRR improvements (on average) against the strongest baselines.

Question 1: *Do the gains come from the bidirectional self-attention model or from the Cloze objective?*

[7] https://github.com/hexiangnan/neural_collaborative_filtering
[8] https://github.com/hidasib/GRU4Rec
[9] https://github.com/graytowne/caser_pytorch
[10] https://github.com/kang205/SASRec
[11] https://github.com/FeiSun/BERT4Rec
[12] What needs to be pointed out is that such low scores for POP are because the negative samples are sampled according to the items' popularity.